中国科学院大学
University of Chinese Academy of Sciences

研究生学位论文中期报告

报告题目 基于 LoongArch 架构的 ART 虚拟机执行引擎适配与优化技术研究

学生姓名 鄢玺 学号 202328013229152

指导教师 张福新 职称 正高级工程师

学位类别 工学硕士

学科专业 计算机科学与技术

研究方向 计算机系统结构

培养单位 中国科学院计算技术研究所

填表日期 2026 年 03 月

中国科学院大学制

报告提纲

一 研究背景与意义 ····· 1

1.1 研究背景 ····· 1

1.2 ART 运行时架构概述 ····· 1

1.3 研究意义 ····· 2

二 国内外研究现状 ····· 2

2.1 Android 运行时发展历程 ····· 2

2.2 虚拟机跨平台移植相关工作 ····· 2

2.3 ART 性能优化研究 ····· 3

三 已完成的研究工作 ····· 3

3.1 解释器实现 ····· 3

3.2 代码生成器实现 ····· 5

3.3 运行时实现 ····· 7

3.4 正确性验证 ····· 9

3.5 系统优化 ····· 10

3.6 性能优化工作：DEX 文件验证器优化 ····· 16

3.7 技术洞察与经验总结 ····· 24

四 研究中遇到的问题 ····· 25

4.1 架构差异导致的兼容性问题 ····· 25

4.2 调试工具链不完善 ····· 26

五 后续研究计划 ····· 26

5.1 短期目标（1-2 个月） ····· 26

5.2 中期目标（3-4 个月） ····· 26

5.3 长期目标 ····· 27

六 总结 ····· 27

一　研究背景与意义

1.1　研究背景

随着国产处理器技术的快速发展，龙芯（LoongArch）架构作为我国自主研发的指令集架构，在服务器、桌面及嵌入式领域得到越来越广泛的应用。龙芯 3A6000 处理器采用 4 核设计、主频 2.5GHz，在 SPEC CPU 2006 等基准测试中已展现出与主流 x86 处理器相当的性能。然而，LoongArch 面临的最大挑战之一是软件生态尚不完善，特别是在移动应用和嵌入式系统方面仍处于起步阶段。

Android 作为全球市场份额超过 70% 的移动操作系统，其核心运行时环境 ART（Android Runtime）直接决定了应用程序的执行效率和用户体验 [1]。ART 采用混合编译策略，结合解释执行、JIT（Just-In-Time）即时编译 [2, 3] 和 AOT（Ahead-Of-Time）预编译三种模式，在安装速度、存储空间和运行性能之间取得平衡。

目前，AOSP 15 官方主流支持 ARM、x86 和 Riscv 等架构，尚未对 LoongArch 架构提供支持。因此，研究如何将 ART 运行时移植到 LoongArch 平台，并针对该平台的架构特点进行性能优化，对于推动国产处理器生态建设具有重要的战略意义。

1.2　ART 运行时架构概述

从技术架构角度，ART 可分为前端和后端两个层次。前端负责将 Java/Kotlin 代码解析为 DEX 字节码，与平台架构无关；后端则需要与底层硬件深度交互，是移植工作的核心。后端主要包含以下组件：

1. **字节码解释器**：ART 提供两种解释器实现。C++ 解释器基于经典的 switch-case 解释循环，采用” 读取-分发-执行” 模式（FDX 循环），可移植性强但效率较低。Nterp 汇编解释器则采用 computed-goto 机制，通过 handler_table_base + (opcode × 128) 直接跳转到对应字节码的处理逻辑，性能可达 C++ 解释器的 4 倍以上。

2. **JIT/AOT 编译器**：基于 Optimizing Compiler 框架，将 DEX 字节码转换为高级中间表示 HIR（High-level Intermediate Representation），经过常量折叠、死代码消除、方法内联等优化后，由架构相关的代码生成器转换为本地机器码 [4]。JIT 采用两级热度检测机制：首阈值触发基础编译（Baseline Compilation），二阈

值触发深度优化编译（Optimized Compilation）。

3. **运行时支持系统**：包括快速路径（art_quick_*）跳板函数、异常处理、垃圾回收（CMC 并发标记-压缩算法）、JNI 调用接口等。Android 15 中默认采用 CMC 垃圾回收器，通过 Linux 内核的 userfaultfd 机制消除读屏障开销。

1.3 研究意义

本研究的主要意义体现在：**（1）推动国产处理器生态建设**：将 Android 系统完整移植到 LoongArch 平台，丰富龙芯处理器的软件生态；**（2）实现核心技术自主可控**：掌握移动操作系统运行时核心技术，包括 JIT 编译器后端、汇编解释器、垃圾回收等关键模块；**（3）积累跨平台移植方法论**：相关经验可推广到其他国产处理器架构。

二 国内外研究现状

2.1 Android 运行时发展历程

Android 运行时经历了从 Dalvik 到 ART 的演进过程。Dalvik 虚拟机由 Dan Bornstein 设计，采用基于寄存器的虚拟机架构 [1]。与 JVM 的栈式虚拟机相比，寄存器式虚拟机的工作方式有所不同：它可以直接对寄存器中的数据进行显式操作，减少了栈操作的开销 [5]，在表示同样程序逻辑的情况下生成比栈式虚拟机更少的指令数量。例如，对于 (a1 + a2) * a3 运算，DEX 仅需 3 条指令，而 Java 字节码需要 6 条（包含额外的访存操作）。

ART 在 Android 5.0 成为默认运行时后，采用了 AOT 编译策略；Android 7.0 重新引入 JIT 编译器，采用混合编译策略：日常使用 JIT 进行少量编译以减少资源占用，系统空闲时进行 AOT 全量编译，在启动速度、存储空间和运行性能之间取得平衡。

2.2 虚拟机跨平台移植相关工作

在虚拟机跨平台移植方面，学术界和工业界积累了丰富经验。OpenJDK 社区已完成 HotSpot JVM 向 LoongArch 架构的移植，包括 C1/C2 JIT 编译器、解释器和 GC 等核心组件，为本研究提供了重要的技术参考。已有针对龙芯 MIPS 平台的 Android 系统移植工作，但这些工作基于 Dalvik 或早期 ART 版本，与最新

的 AOSP 15 存在较大差异。

2.3 ART 性能优化研究

ART 性能优化研究涵盖多个方向。在 JIT 编译器优化方面，Samsung 在 ART 的 JIT 编译过程中探索了结合机器学习的配置感知型优化策略，使用随机森林模型对运行时性能剖面数据建模；复旦大学团队提出 JavART，通过规则驱动的 JIT 编译器直接生成高质量本地代码，跳过解释执行阶段。在编译优化技术方面，主要包括常量折叠、死代码消除、栈上替换（OSR）、内联优化等经典技术。

三 已完成的研究工作

本研究基于 AOSP 15 源码，围绕 LoongArch 平台的 ART 运行时（简称 ART-TOL）移植与优化开展工作。采用从易到难的实现策略：首先实现可移植性最强的 C++ 解释器，验证正确性后再实现 Nterp 汇编解释器和 JIT 编译器后端。整体测试环境为龙芯 3A6000 处理器（4 核、2.5GHz、16GB 内存）、Android 15 系统、Linux 6.6.46 内核。

3.1 解释器实现

3.1.1 C++ 解释器适配

C++ 解释器采用经典的字节码解释循环（" 读取-分发-执行" 模式），通过 switch-case 将每条 DEX 指令分发到对应的操作函数。由于 ART 为寄存器式虚拟机，所有操作都在虚拟寄存器（ShadowFrame）中执行。核心执行流程包括：

1. **指令预取与追踪**：从 pc 获取下一条指令，记录 dex_pc 到 ShadowFrame，调用 TraceExecution 记录执行信息。

2. **指令解析与执行**：获取指令的 16 位数据，封装为 InstructionHandler 对象，根据 opcode 分发到对应处理函数（如 OP_ADD_INT、OP_NEG_INT 等）。

3. **异常处理**：通过 HandlePendingException() 查找 catch handler；若未找到则返回由调用方处理。

尽管解释器本身使用 C++ 编写，但在移植至龙架构时仍需处理若干平台相关细节。首先，需严格遵循 LoongArch 的调用约定，包括参数传递方式（$a0-$a7

传参、$fa0-$fa7 浮点传参）、返回值处理和栈帧对齐规则。其次，解释器依赖若干汇编实现的辅助跳板函数，如 art_quick_invoke_static_stub，在调用静态 Java 方法时需要：创建调用栈帧和上下文，从寄存器或内存读取参数，根据 ArtMethod* 调用对应方法，最后返回结果并恢复上下文。

3.1.2 Nterp 汇编解释器移植

Nterp 采用 computed-goto 机制对字节码逐条执行，将每条字节码的汇编解释逻辑限制在 128 字节内，通过 xIBASE + 128 × opcode 直接跳转到对应处理逻辑，显著减少传统解释器的分支判断开销。

Nterp 调用规范 针对 LoongArch 架构，本文定义了专用的 Nterp ABI 规范，如表 1所示：

表 1 Nterp 调用规范（LoongArch）

宏名	寄存器	用途
xSELF	$s1/$r24	线程指针
xFP	$s2/$r25	解释器帧指针，访问局部变量与参数
xPC	$s3/$r26	解释器程序计数器，用于指令获取
xINST	$s4/$r27	当前指令编码
xIBASE	$s5/$r28	解释器指令跳转基址
xREFS	$s6/$r29	Dex 寄存器中的对象引用基址

Nterp 入口与栈帧设置 使用 ExecuteNterpImpl 作为 Nterp 入口函数，当 ART 判断当前环境支持 Nterp 后跳入此入口进行初始化：建立栈帧、将参数放入对应寄存器、根据参数类型选择不同的解析路径。对于无参方法（s8 == 0），直接进入 .Lentry_go 分支跳过参数设置流程。对于有参方法，根据 flags 判断是否可走 fast-path（全部参数为引用类型时通过 setup_ref_args_and_go 宏快速完成）。

对于含基本类型参数的方法，需通过 NterpGetShorty 动态获取 shorty 描述符，进入三阶段参数重定向：store_gpr_to_vreg 处理 a1-a7 的整型参数写入，store_outs_to_vregs 拷贝超出寄存器范围的参数，store_fpr_to_vreg 处理 fa0-fa7 浮

点参数。完成后将 artNterpAsmInstructionStart（跳转表基址）存入 xIBASE，运行 START_EXECUTING_INSTRUCTIONS 宏开始主循环。

字节码指令实现 谷歌为 ART 预留了 256 条 DEX 字节码位置，除少数占位指令外均需使用 LoongArch 汇编实现。以 instance-of 指令（22c 格式）为例，其语义为 vA = (vB instanceof type@CCCC) ? 1 : 0，实现步骤如下：

1. 使用 srli.w 和 andi 解析 vA、vB 字段及类型索引 CCCC
2. 通过 FETCH_FROM_THREAD_CACHE 从线程本地缓存快速加载目标类型
3. 检查对象寄存器是否为 null，从对象结构提取类信息进行比较
4. 类型一致则返回值设为 1，跳转至指令调度器执行下一条字节码
5. 否则跳转到 slow path 执行完整类型层级判断

由于每个字节码处理逻辑限制在 128 字节，超出时需使用额外的 HELPER 函数处理。相比 X86 的变长指令，LoongArch 的定长 4 字节指令在阅读上更清晰，但在复杂字节码解释上需要更多指令，因此 HELPER 函数的使用更为频繁。

异常处理与栈帧切换 Nterp 采用效率最大化设计，近乎接近编译执行，抛弃了对自身状态的定期异常检查。当遇到无法处理的异常（如除零异常 DivideByZero）时，通过 QuickDeliverException 抛出异常，跳转回 C++ 解释器处理。Nterp 与 C++ 解释器共享统一的栈帧布局，切换时无需重新分配或保存新栈帧结构，极大提升了切换效率。

3.2 代码生成器实现

代码生成器负责将中间码转换为龙架构下的本地可执行指令，直接影响应用的运行效率。ARTOL 的代码生成器包含汇编器、反汇编器、链接器和指令生成器四个子模块。

3.2.1 汇编器与反汇编器

汇编器负责将指令生成器输出的指令编码为符合龙架构语义与编码规则的机器指令。LoongArch 采用定长 32 位指令格式，包含九种典型编码格式：3 种不

含立即数格式（2R、3R、4R）和 6 种含立即数格式（2RI8、2RI12、2RI14、2RI16、1RI21、I26）。

核心机制采用结构化的编码辅助函数，如 Emit2RI12(opcode, imm12, rj, rd)，将输入参数打包为合法的 32 位指令字：

```
template <typename Reg2, typename Reg1>
void Emit2RI12(uint32_t opcode, int32_t imm12,
                Reg2 rj, Reg1 rd) {
    DCHECK(IsUint<10>(opcode));
    imm12 = imm12 & 0xfff;
    uint32_t encoding = opcode << 22 |
        static_cast<uint32_t>(imm12) << 10 |
        static_cast<uint32_t>(rj) << 5 |
        static_cast<uint32_t>(rd);
    Emit(encoding);
}
```

反汇编器基于”解码表驱动”与”位段匹配”相结合的机制实现，通过多级查找表（opcode → funct3 → funct7 → handler）快速映射指令处理路径。

3.2.2 汇编器测试框架

汇编器测试框架用于验证每条指令在不同参数组合下生成的二进制是否正确。测试集需覆盖所有基础指令，并通过极端参数值（立即数最大最小值、寄存器号溢出、符号位截断等）揭示潜在的编码错误。

测试原理：将同一段待测代码分别通过 ARTOL 汇编器和标准 Clang 编译器汇编，逐字节比对两者生成的二进制。若完全一致，则证明汇编器实现正确。如大位移后向跳转测试：

```
TEST_F(AssemblerLOONGARCH64Test, BlRdBackward2MiB) {
    TestBlRdBackward("BlRdBackward2MiB",
                     2 * MB, "1", GetPrintBlRd());
}
```

该测试构造了指令距离达 2MiB 的后向跳转，覆盖 bl 指令在龙架构上的最大跳转距离边界。

3.2.3 链接器

链接器处理分支跳转指令的目标地址计算与重定位。对于 beq、bl 等跳转指令，在代码生成阶段无法直接给定有效机器指令，采用” 先占位、后回填” 机制：

1. **占位阶段**：跳转目标地址设置为特殊值 0xffffffff，记录该指令在 buffer_ 中的相对地址，所有跳转指令放入 branches_ 队列

2. **绑定阶段**：当 label 被 Bind 指令给定后，从 buffer_ 获取相对 PC 位置，遍历分支队列中指向该 label 的所有 branch，调用 Resolve 设置正确的 target_ 地址

3. **回填阶段**：FinalizeCode 阶段调用 EmitBranches 对所有分支指令进行重写，根据分支距离选择短跳转（beq）、中跳转（jirl）或长跳转（pcaddu18i + jirl）

3.2.4 指令生成器

ART 在 DEX 字节码与汇编代码之间引入高级中间表示 HIR，保留程序的控制流和数据依赖信息，支持常量传播、死码消除等高级优化。每个方法的 HIR 构成一个 HGraph，其中节点（HInstruction）对应程序操作单元，边表示数据依赖与控制依赖。

指令生成器通过 CodeGeneratorLOONGARCH64 类实现，按块遍历 HGraph，对每条 HIR 指令调用对应生成函数（如 VisitAdd、VisitCompare），结合寄存器分配和调用约定，转换为汇编指令序列。例如 HAdd 可能被映射为 add、addi 或 li + add 的组合，需根据输入数据类型做不同处理。

同时实现 SlowPath 机制支持异常路径：每处理一个 HNullCheck 节点时，主路径生成非空性测试指令（bnez），在 null 分支绑定 NullCheckSlowPathLOONGARCH64 实例。该慢路径在最终指令排布时插入到主路径后续，调用运行时入口 kQuickThrowNullPointer 抛出 NullPointerException。

3.3 运行时实现

运行时系统提供快速路径、异常处理和垃圾回收支持，是保障 ARTOL 正确运行的核心。

3.3.1 快速路径（Quick Entrypoints）

ART 广泛使用 art_quick_* 前缀命名的快速路径函数，为方法调用、对象分配、异常处理等高频操作提供高效支持。系统初始化阶段通过 DefaultInitEntryPoints 将通用逻辑层函数指针与架构相关汇编符号绑定，ARTOL 在 entrypoints_init_loongarch64.cc 中调用此注册函数，并在 quick_entrypoints_loongarch64.S 中实现。

以通用 JNI 调用为例，主要由 art_quick_generic_jni_trampoline 和 art_jni_dlsym_lookup_stub 构成。前者在 native 方法初次调用时触发延迟解析，构建 JNI 调用栈帧（预留 5KB 栈空间），执行 Java 到 native 的线程状态切换后跳转至 native 函数。后者负责通过 dlsym 在动态库中查找匹配函数并缓存至 ArtMethod。

3.3.2 异常处理

ARTOL 支持两种异常处理路径：

1. **解释器同步异常**：运行于用户态，采用同步判定与延迟跳转。解释器执行时对非法操作（null 解引用、数组越界等）进行逻辑判断，通过 Thread::SetException() 记录异常对象，跳转至 art_deliver_pending_exception，最终调用 artDeliverPendingExceptionFromCode 完成栈展开，通过 LoongArch64Context::DoLongJump 跳转到 catch 语句块。

2. **编译代码异步异常**：JIT/AOT 编译代码可能触发 SIGSEGV 信号。ART 初始化时通过 FaultManager::Init() 向内核注册自定义信号处理器，当编译代码触发内核页保护机制时，通过 fault_handler_loongarch64.cc 提取故障现场信息（PC 值、32 个通用寄存器、fault address），判断是否为受控异常（栈溢出检查或空检查），修改 PC 为对应异常处理函数后返回用户态完成处理。

3.3.3 垃圾回收支持

Android 15 默认使用 CMC（Concurrent Mark-Compact）垃圾回收器 [6, 7]，通过 Linux 内核的 userfaultfd 机制消除读屏障开销。ARTOL 需在各种 Runtime 函数中插入对垃圾回收的处理。以 art_quick_aput_obj 为例，需在对象引用写入后更新卡表（Card Table）以确保并发标记正确性：

1. 执行三次 Read Barrier 读取目标数组与待写入对象的类元信息

2. 类型一致时使用 stx.w 指令完成 32 位堆引用存储

3. 读取线程局部的卡表地址，使用数组对象地址右移后的结果定位对应卡片字节

4. 由于龙架构为弱内存模型，通过 amoswap 与 dbar 确保卡表写入的原子性与多线程可见性

3.4 正确性验证

3.4.1 Native 端测试

使用 ART 自带的 m test-art-host-gtest 命令执行 Host 侧 GTest 测试集，覆盖 DEX 字节码生成、DEX 文件验证、ART 命令行测试、libartbase 库等多个子系统。通过所有测试用例确保 ARTOL 实现未对 ART 主体结构造成影响。测试结果包括：art_libdexfile_support_tests、art_cmdline_tests、art_compiler_tests、art_runtime_tests 等均通过。

3.4.2 Target 端测试

Target 端测试在真实目标设备上验证 ARTOL 的 Java 程序执行正确性。测试框架将测试用例编译为 DEX 文件，连同 ARTOL 的动态库（libart.so、libartd-compiler.so、libartbase.so、libc.so 等）推送到设备的 /data/local/art-test-chroot 目录，形成隔离的测试沙箱。

使用命令 art/test.py --target -r --interpreter --64 执行测试，分别在三种执行模式下完成验证：

表 2 Target 端测试结果

执行模式	通过测试数	通过率
C++ 解释器	1033/1033	100%
Nterp 解释器	1033/1033	100%
JIT 编译器	1033/1033	100%

3.4.3 libcore 测试集

libcore 测试集覆盖 Java 核心 API 的行为语义验证，包括集合框架、输入输出、并发机制、数学运算、加解密、网络通信等模块。使用 art/tools/run-libcore-tests.sh --mode=device --variant=X64 运行全部 146 个测试包，所有测试用例均通过（部分测试因架构差异被自动跳过）。

3.4.4 调试框架

针对 ARTOL 的三种执行方式，分别搭建了不同的调试框架：

1. **差分测试（Diff Test）**：将同一段 Java 程序分别在官方 X86 架构 ART 和 ARTOL 的 C++ 解释器中运行，通过 TraceExecution 函数输出每条字节码执行后的虚拟机状态，逐条比对虚拟寄存器内容与对象引用，定位首次出现偏差的位置。

2. **逐条指令迭代验证**：Nterp 汇编解释器采用逐条指令添加的方式，在 CanMethodUseNterp() 中引入过滤逻辑屏蔽未支持指令，每次添加新指令后与 C++ 解释器结果比对验证。

3. **CFG 可视化**：JIT 编译器通过 --dump-cfg=result.txt 导出控制流图，记录每个方法的基本块、HIR 节点、生成的汇编指令及其地址映射，便于在崩溃时通过 PC 地址逆推触发问题的语义节点。

3.5 系统优化

在完成基本移植并验证正确性后，本文围绕高频路径和热点方法执行效率，实施了多项优化策略。

3.5.1 JNI 编译优化

理论背景：JNI 调用开销的本质 JNI（Java Native Interface）是 Java 与本地代码（C/C++）交互的标准接口。在 ART 中，JNI 调用涉及复杂的跨语言边界处理，其开销主要来源于以下几个方面：

（1）线程状态转换开销：ART 维护线程的运行状态（Runnable、Native、Suspended 等）。当 Java 代码调用 native 方法时，线程需从 Runnable 状态切换到 Native 状态；返回时再切换回来。状态转换涉及：

- 修改 Thread::state_ 字段
- 检查是否有挂起请求（Suspend Request）
- 同步内存屏障确保可见性

每次状态切换约消耗 15-30 个 CPU 周期。

（2）JNI 参数封装开销：JNI 规范要求将 Java 对象引用封装为" 间接引用"（jobject），存储在 JNI 本地引用表（Local Reference Table）中。这种封装的目的是：

- 防止 native 代码直接持有堆对象指针（GC 可能移动对象）
- 提供统一的对象访问接口

但每次封装/解封装需要表查找和指针间接访问，累积开销可观。

（3）通用跳板函数开销：原有实现中，所有 JNI 调用都经过 art_quick_generic_jni_trampoline，该函数需要：

- 动态解析方法签名（shorty）确定参数类型
- 根据签名将参数从 Java 调用约定转换为 C 调用约定
- 构建完整的 JNI 栈帧（预留 5KB 空间）
- 处理同步方法的 monitor 操作

（4）优化思路：针对上述开销，Android 8.0 引入了两种优化注解：

表 3 @FastNative 与 @CriticalNative 对比

特性	@FastNative	@CriticalNative
支持 JNIEnv*	是	否
支持传递 JObject	是	否
线程状态切换	是（简化）	否
GC 安全点支持	是	否
适用场景	需要访问 Java 对象	纯数值计算
典型延迟	30-50 ns	20-30 ns

@FastNative 优化原理：保留 JNIEnv* 访问能力，但简化线程状态转换路径，跳过部分安全检查。@CriticalNative 优化原理：完全绕过 JNI 框架，native 方法直接接收原始参数，调用开销接近纯 C 函数调用。

具体实现 为 JNI 调用实现专用编译器，通过 ArtJniCompileMethodInternal 函数生成定制化跳板代码，核心流程包括 18 个步骤：

1. 构建方法调用栈帧（@CriticalNative 额外分配参数传递空间）
2. 写入栈顶指针到 Thread::TopOfManagedStack（@CriticalNative 跳过）
3. 为出参扩展栈帧
4. 进行读屏障判定（启用 Read Barrier 时检查 jclass 是否需 slow path）
5. 调用 JniMethodStart 进行线程状态转换
6. 设置并传递 Java 参数（@CriticalNative 直接以机器本地类型传递）
7. 处理 static 方法的 jclass 参数
8. 创建 JNIEnv* 参数（@CriticalNative 省略）
9. 使用 JNI 入口点偏移量调用 native 代码
10. 返回值位宽对齐（窄类型执行零扩展或符号扩展）
11. 保存返回值（@CriticalNative 跳过）
12. 调用 JniMethodEnd 恢复线程状态
13. 返回值重载
14. 参数使用后调整栈帧
15. 异常处理
16. 移除栈帧并恢复 callee-saved 寄存器
17. 静态调用的读屏障慢路径设置
18. 完成代码生成

性能数据 使用 ART 自带的 JniPerfBenchmark 测试程序，在 JIT 模式下测量不同类型 JNI 方法的单次调用延迟：

表 4 不同 JNI 方法调用时间

JNI 方法类型	调用延迟
Generic Call	121 ns
@FastNative	38 ns
@CriticalNative	27 ns

相比通用 JNI 方法，@FastNative 和 @CriticalNative 的调用延迟分别降低

68.6% 和 **77.7%**。

3.5.2 内联优化

理论背景：方法调用开销与内联收益分析 在虚拟机执行模型中，方法调用是一项开销显著的操作。以 ART 的 JIT 编译器为例，一次普通的 Java 方法调用涉及以下步骤：

（1）方法调用开销分解：

- **参数准备**：将参数压入栈或放入寄存器（2-8 个 CPU 周期）
- **方法解析**：对于虚方法，需通过 vtable 查找实际方法地址（3-5 周期，可能 cache miss）
- **栈帧构建**：分配新栈帧、保存返回地址、调整帧指针（5-10 周期）
- **跳转与返回**：分支预测可能失败，流水线刷新（10-20 周期惩罚）
- **栈帧销毁**：恢复调用者上下文（3-5 周期）

单次方法调用的累积开销约为 **20-50 个 CPU 周期**。

（2）内联优化的本质：内联（Inlining）将被调用方法的代码直接嵌入调用点，消除上述所有调用开销。更重要的是，内联后编译器可进行**跨过程优化**：

- **常量传播**：调用点的常量参数可传播到方法体内
- **死代码消除**：基于常量条件的分支可被消除
- **公共子表达式消除**：跨调用边界的重复计算可被识别
- **寄存器分配优化**：消除参数传递的寄存器移动

（3）Intrinsic 方法：超级字节码：ART 对特定的高频库方法（如 Math.abs()、String.charAt()）实现了 Intrinsic 优化。与普通内联不同，Intrinsic 直接将 Java 方法语义映射为 1-3 条机器指令，完全跳过中间表示（HIR）生成过程。这种优化可视为” 超级字节码”——用单条指令替代完整的方法调用语义。

以 Math.sqrt(double) 为例，其优化效果对比如表 5所示：

（4）内联决策的权衡：内联并非无代价——过度内联会导致代码膨胀（Code Bloat），增加指令缓存压力。ART 的内联策略考虑：

- 方法体大小（通常阈值为 100 DEX 指令单元）
- 调用深度（避免递归内联）
- 调用点热度（优先内联热路径）

表 5 Math.sqrt() 调用方式开销对比

调用方式	指令数	周期数	访存次数
解释执行	~50	~200	8+
JIT 编译（无内联）	~15	~40	2
Intrinsic 优化	1	4-8	0

- 方法属性（final、private、static 方法更易内联）

具体实现 以 Math.sqrt(double) 为例，常规编译流程会翻译为静态方法调用，涉及参数压栈、方法解析、调用栈构建等。ARTOL 识别该方法为 Intrinsic 后：

```
void IntrinsicCodeGeneratorLOONGARCH64::
    VisitMathSqrt(HInvoke* invoke) {
    LocationSummary* locations = invoke->GetLocations();
    FRegister in = locations->InAt(0)
        .AsFpuRegister<FRegister>();
    FRegister out = locations->Out()
        .AsFpuRegister<FRegister>();
    __ FSqrt_D(out, in);  // 单条指令
}
```

原本需要多个字节码实现的平方根逻辑被替换为单条 fsqrt.d 指令。

性能数据 使用 CaffeineMark 基准测试，在开启与关闭内联优化（--inline-max-code-units=0）两种配置下对比：

- 整体得分提升 **35.37%**
- Method 测试项（方法调用频率高度相关）提升近 **5 倍**

3.5.3 OSR 栈上替换

理论背景：动态编译中的"热循环困境" 栈上替换（On-Stack Replacement, OSR）是 JIT 编译器中的核心优化技术，其诞生源于动态编译系统面临的一个根本性挑战——**热循环困境**。

（1）问题描述：考虑以下典型的基准测试代码：

```
public static void main(String[] args) {
    long sum = 0;
    for (int i = 0; i < 10_000_000; i++) {
        sum += compute(i);  // 热循环
    }
    System.out.println(sum);
}
```

在传统的方法级 JIT 编译中，编译器以**方法**为编译单位。当 main() 方法开始执行时：

1. 解释器开始逐条执行字节码
2. 循环执行若干次后，JIT 探测到 main() 是热点方法
3. JIT 编译器异步编译 main() 方法
4. 编译完成后，main() 的优化版本可用
5. **问题**：此时 main() 正在执行中，无法切换到编译版本！

由于 main() 仅被调用一次，编译完成的优化代码**永远不会被执行**。对于包含长时间运行循环的方法，这种”编译了却用不上”的情况严重限制了 JIT 的优化收益。

（2）OSR 的核心思想：OSR 允许在方法**执行过程中**，从解释执行切换到已编译的本地代码。其关键挑战是**执行状态迁移**：

- **程序计数器映射**：解释器的 dex_pc 需映射到编译代码的 native_pc
- **局部变量迁移**：解释器的虚拟寄存器（ShadowFrame）需迁移到编译代码的物理寄存器/栈位置
- **栈帧重建**：解释器栈帧与编译代码栈帧布局不同，需要转换

（3）OSR 入口点（Entry Point）：并非编译代码的任意位置都能作为 OSR 入口。ART 在编译时会在**循环回边**（Back Edge）处生成 OSR 入口点，并记录该点的 **StackMap**——一张描述”此刻哪些虚拟寄存器对应哪些物理位置”的映射表。StackMap 是 OSR 状态迁移的关键数据结构。

（4）OSR 的两个方向：

- **上行 OSR（Tier-up）**：解释执行 → 编译执行。本节讨论的主要场景。
- **下行 OSR（Deoptimization）**：编译执行 → 解释执行。当编译代码的优化假设失效时（如类层次结构变化），需回退到解释器执行。

具体实现 OSR 流程分为准备阶段和跳转阶段：

1. **准备阶段**：由 Jit::MaybeDoOnStackReplacement 驱动，触发条件包括：当前解释器执行位于热点代码段、当前线程未接近栈底、目标方法已由 JIT 编译且存在 OSR StackMap。Jit::PrepareForOsr 根据当前 dex_pc 定位编译代码中的入口偏移，构造 OsrData 结构（栈帧大小、编译代码地址、虚拟寄存器状态映射）。

2. **跳转阶段**：跳入 art_quick_osr_stub 进行栈帧替换，保存当前线程信息，动态调整 sp 指针为目标方法预留栈空间，通过循环将当前栈数据迁移至新栈帧，最后跳转到 JIT 编译代码的中间执行点。

```
.Losr_loop:
    addi.d  t0, t0, -8
    ld.d    t2, t0, 0
    addi.d  t1, t1, -8
    st.d    t2, t1, 0
    bne     t1, sp, .Losr_loop
```

性能数据 使用 ART 测试集中包含 2,000,000 次循环的测试程序，经 10 次测试取平均值：

- 优化前：16.229s
- 优化后：12.488s
- 效率提升：**23.05%**

3.6 性能优化工作：DEX 文件验证器优化

在完成基本移植后，通过 Profiling 分析发现，DEX 文件验证器（DexFileVerifier）是应用启动过程中的主要性能热点之一。每次应用启动都需要验证 DEX 文件的格式正确性和安全性，验证延迟直接影响冷启动时间。

3.6.1 理论背景：缓存层次结构与数据访问模式

（1）现代 CPU 缓存层次结构：现代处理器采用多级缓存架构来弥合 CPU 与内存之间的速度差距。以龙芯 3A6000 为例：

表 6 龙芯 3A6000 缓存层次结构

缓存级别	容量	延迟	带宽
L1 数据缓存	64 KB/核	3-4 周期	128 GB/s
L2 缓存	256 KB/核	10-12 周期	64 GB/s
L3 缓存	16 MB（共享）	30-40 周期	32 GB/s
主存（DDR4）	16 GB	80-120 周期	25 GB/s

L1 缓存访问与主存访问的延迟差距可达 **20-30 倍**。因此，优化的核心目标是**最大化缓存命中率**。

（2）缓存友好的数据结构设计原则：

- **空间局部性**：连续访问的数据应在内存中相邻存储。数组和 Vector 天然满足此特性，而链表和 HashMap（链式冲突处理）则破坏空间局部性。
- **时间局部性**：近期访问的数据可能很快再次被访问，应保持在缓存中。
- **数据紧凑性**：减少每个元素的内存占用，使更多元素能装入缓存行（通常 64 字节）。

（3）HashMap vs. Sorted Vector 的真实性能对比：传统算法分析认为 HashMap 查找为 O(1)，优于 Vector 二分搜索的 O(log n)。然而，这种分析忽略了**常数因子**和**缓存效应**：

对于 DEX 验证器场景（元素数量 < 1000），Vector 的实际性能优于 HashMap，原因包括：

- 二分搜索的 10 次比较可在 2-3 次缓存行加载内完成（每行 64 字节 / 8 字节 = 8 个元素）
- HashMap 的哈希计算本身消耗 5-10 周期
- 链式冲突处理的指针跳转几乎必然 cache miss

（4）分支预测与查表法：现代 CPU 采用分支预测器来推测性执行。当预测失败时，流水线需要刷新，惩罚约为 **15-20 个周期**。对于 switch-case 结构：

表 7 HashMap 与 Sorted Vector 性能对比（1000 个元素）

指标	HashMap	Sorted Vector
理论复杂度	O(1)	O(log n) ≈ 10
每元素内存占用	24-32 字节	8 字节
缓存行利用率	~12%（仅 Key+Value）	100%
查找访存次数	2-4 次（hash + 链）	3-4 次（二分）
预取友好性	差（随机访问）	好（顺序扫描模式）
实测查找延迟	~45 ns	~25 ns

- 分支数量 ≤ 4：编译器通常生成条件跳转链，预测准确率高
- 分支数量 > 4：编译器生成跳转表或决策树，预测准确率下降
- UTF-8 解码的 switch-case 有 5+ 分支，且分支概率不均匀，预测失败率高

查表法用**内存访问**替代**分支判断**，利用 L1 缓存的低延迟（3-4 周期）规避分支预测失败的高惩罚（15-20 周期）。

3.6.2 性能分析方法

本研究采用以下工具和方法进行性能分析：

1. **simpleperf**：Android 平台的性能分析工具，支持 CPU 采样和硬件性能计数器。针对 LoongArch 和 ART 环境的特点，采用 –fp（Frame Pointer）方式进行栈展开，相比 DWARF 方式在 JIT 代码上更加稳定。

2. **火焰图（FlameGraph）**：可视化函数调用开销分布，快速定位热点函数。

3. **Google Test 基准测试**：使用 ART 自带的单元测试框架进行微基准测试，每组测试迭代 50-200 次，统计平均延迟、P50/P99 百分位延迟和吞吐量。

3.6.3 问题分析

通过 Profiling 分析，发现原始 DexFileVerifier 实现存在以下性能问题：

1. **频繁的边界检查**：每次调用 EndOfFile() 都需要通过 Begin() + size_ 计算文件边界，在字符串验证等热路径中产生大量重复计算。

2. **HashMap 缓存不友好**：验证器使用 std::unordered_map 存储偏移量到类型的映射关系。HashMap 的问题包括：

- 每个元素需要 24-32 字节（Key 4B + Value 2B + 对齐 + 指针 + Hash 值）
- 数据分散存储，L1/L2 缓存命中率低
- 每次查找需要计算哈希值
- 链式冲突处理需要额外指针跳转

3. **UTF-8 字符串处理效率低**：原代码使用 switch-case 判断 UTF-8 字节类型，存在 5 个以上分支，CPU 分支预测失败率高。

3.6.4 第一阶段优化：基础性能优化

针对上述问题，第一阶段实施了以下优化措施：

文件边界缓存 在构造函数中预计算文件边界地址，避免在热路径中重复计算：

```
// 构造函数中预计算文件边界
file_begin_ = offset_base_address_;
file_end_ = offset_base_address_ + size_;

// 热路径中直接使用缓存值
inline const uint8_t* EndOfFile() const {
    return file_end_;
}
```

HashMap 预分配 根据 DEX 头部信息预估数据规模并预分配容量，消除运行时 rehash 开销：

```
// 根据 DEX header 预估容量
size_t estimated_entries =
    header_->class_defs_size_ +
    header_->method_ids_size_ +
    header_->field_ids_size_;
defined_classes_.reserve(estimated_entries);
```

热函数内联　将频繁调用的小函数标记为 inline，消除函数调用开销：

```
inline const uint8_t* EndOfFile() const {
    return file_end_;
}
inline size_t Offset() const {
    return ptr_ - file_begin_;
}
```

第一阶段优化效果　如图 1所示，第一阶段优化后，平均延迟降低 **18.1%**，P99 延迟从 115μs 降至 98μs。

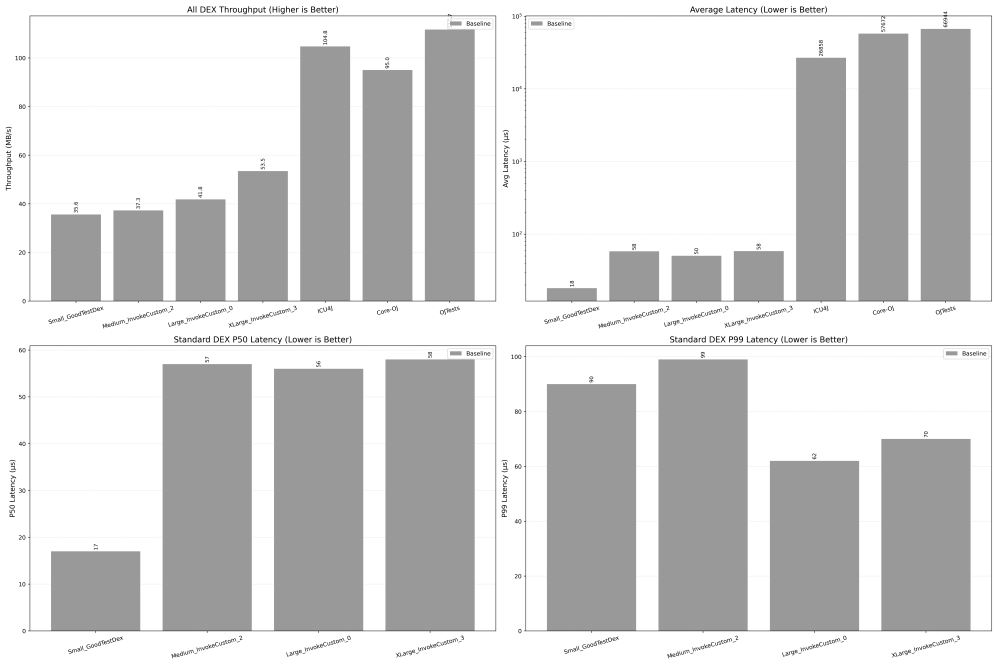

图 1 基线性能测试结果

3.6.5　第二阶段优化：数据结构优化

第二阶段针对 HashMap 缓存不友好的问题，将其替换为排序 Vector + 二分查找。

优化原理　传统观点认为 HashMap 查找 O(1) 优于 Vector 二分搜索 O(log n)。然而，对于中等规模数据（< 1000 条），实际情况恰恰相反：

- **内存紧凑**：Vector 连续存储，每个元素仅占 8 字节（Key 4B + Value 2B + 对齐 2B），远小于 HashMap 的 24-32 字节。
- **缓存友好**：现代 CPU L1 缓存 32-64KB，可容纳 4000-8000 个 Vector 元素。二分搜索访问模式规律，CPU 硬件预取效率高。
- **插入有序**：DEX 验证器按地址顺序扫描文件，偏移量天然递增，可直接追加无需排序。

实现方式

```
// 替换数据结构
// 旧: std::unordered_map<uint32_t, uint16_t>
// 新: std::vector<std::pair<uint32_t, uint16_t>>

// 插入时保持有序（实际上天然有序，直接追加）
offset_to_type_map_.push_back({offset, type});

// 查找使用二分搜索
auto it = std::lower_bound(
    offset_to_type_map_.begin(),
    offset_to_type_map_.end(),
    offset,
    [](const auto& pair, uint32_t off) {
        return pair.first < off;
    });
```

第二阶段优化效果 如图 2所示，第二阶段优化后：

- 小文件（676B）延迟降低 **32.8%**
- 大文件吞吐量从 44.10 MB/s 提升至 57.37 MB/s，增长 **30.1%**

3.6.6 第三阶段优化：字符串处理优化

第三阶段针对 UTF-8 字符串验证进行优化。DEX 文件中约 80% 的字符串为 ASCII 编码（类名、方法名、包名），可以建立快速路径。

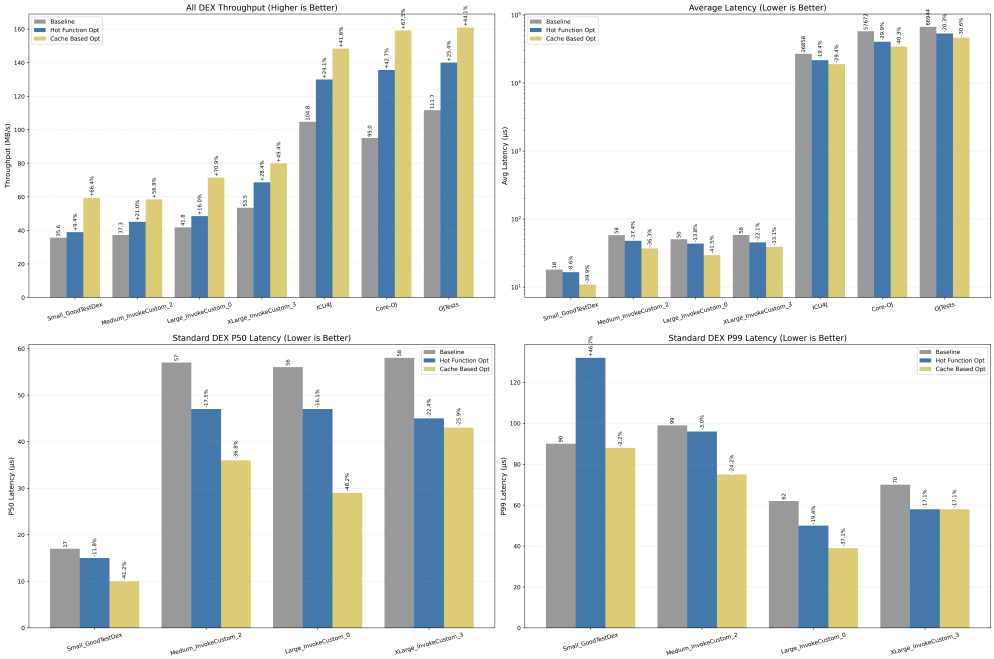

图 2 数据结构优化后性能测试结果

ASCII 快速路径 使用位运算快速判断 ASCII 字符，避免完整 UTF-8 解码：

```
// ASCII 快速路径
while (*utf8_1 != '\0' && *utf8_2 != '\0') {
    uint8_t c1 = *utf8_1;
    uint8_t c2 = *utf8_2;

    // 两个字符都是 ASCII (< 0x80)
    if ((c1 | c2) < 0x80) {
        if (c1 != c2) return c1 - c2;
        utf8_1++;
        utf8_2++;
        continue;  // 快速路径
    }
    // 非 ASCII 字符，回退到完整解码逻辑
    ...
}
```

UTF-8 验证查表法 将 switch-case 分支判断替换为查找表，消除分支预测失败开销：

```
// 预计算 UTF-8 字节类型查找表
static constexpr uint8_t kUtf8ByteTypeTable[16] = {
    0b0001, 0b0001, ..., 0b1000
};

// 使用查表替代 switch-case
inline uint8_t GetUtf8ByteType(uint8_t byte) {
    return kUtf8ByteTypeTable[byte >> 4];
}
```

第三阶段优化效果 如图 3所示，第三阶段优化对大文件效果显著：

- ICU4J（2.9MB）吞吐量从 148.34 MB/s 提升至 164.57 MB/s，改进 **10.9%**
- P99 延迟从 91μs 降至 88μs

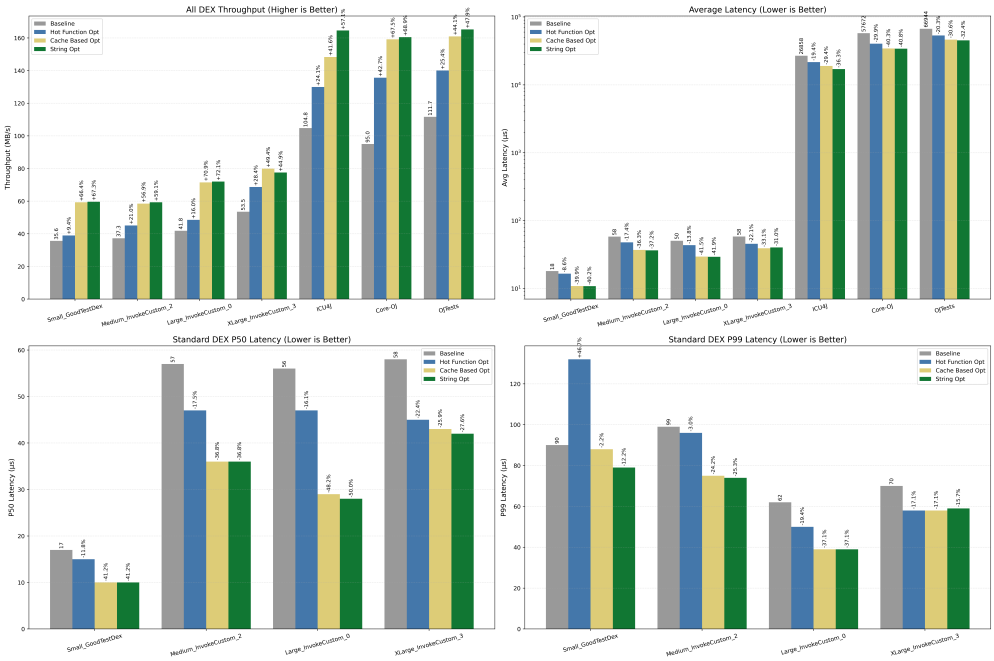

图 3 字符串处理优化后性能测试结果

3.6.7 优化效果总结

表 8和表 9分别展示了优化前后的性能对比和各阶段贡献分析。

表 8 DEX 验证器优化效果总结

测试项	基线	优化后	改进幅度
基准延迟 (GoodTestDex 676B)	19.82 µs	10.85 µs	-45.3%
小文件吞吐量	37.25 MB/s	62.13 MB/s	+66.8%
大文件吞吐量 (ICU4J 2.9MB)	104.75 MB/s	164.57 MB/s	+57.3%
P99 延迟	115 µs	88 µs	-23.5%

表 9 各优化阶段贡献分析

阶段	主要优化	延迟改进	吞吐量改进
Phase 1	边界缓存、预分配、内联	-18.1%	+24.1%
Phase 2	HashMap → Vector	-32.8%	+30.1%
Phase 3	UTF-8 查表、ASCII 快速路径	-0.5%∼-9.9%	+10.9%
累积		**-45.3%**	**+57.3%**

真实 AOSP DEX 文件测试结果 表 10展示了在真实 AOSP 系统库 DEX 文件上的测试结果。

表 10 真实 AOSP DEX 文件测试结果

DEX 文件	大小	基线延迟	优化后延迟	改进
ICU4J	2.9 MB	26858 µs	17096 µs	-36.3%
Core-OJ	5.5 MB	57672 µs	34147 µs	-40.8%
OJTests	7.5 MB	66944 µs	45256 µs	-32.4%

3.7 技术洞察与经验总结

通过本次优化工作，总结出以下技术洞察：

1. **数据结构选择需考虑缓存效应**：对于中等规模数据（< 1000 条），缓存友好的 Vector 实际性能优于理论更优的 HashMap。这类似于编译器优化中用 computed goto 替代 switch 的技巧。

2. **快速路径的价值**：即使只覆盖 70-80% 的场景，在热路径上建立快速分支也能带来显著收益。ASCII 快速路径在字符串密集场景下累积收益可观。

3. **分支预测的影响**：查表法虽然理论复杂度与 switch-case 相同（都是 O(1)），但实际性能可快 15-20%，原因是消除了分支预测失败的惩罚。

四　研究中遇到的问题

4.1　架构差异导致的兼容性问题

在移植过程中，LoongArch 与已支持架构（如 ARM64、x86_64）之间的差异带来了诸多挑战：

1. **内存序模型差异**：LoongArch 采用弱内存模型，ARM 的 Load-Acquire/Store-Release 语义需使用 dbar 指令显式实现。在垃圾回收的卡表更新中，需通过 amoswap 与 dbar 确保原子性与多线程可见性，否则标记线程可能遗漏脏卡导致存活对象被错误回收。

2. **调用约定差异**：LoongArch 的浮点参数传递使用 $fa0-$fa7 寄存器，与整数参数混合时规则与 ARM64 不同。在 JNI 编译器和 Nterp 入口函数中需分别处理整型参数（store_gpr_to_vreg）和浮点参数（store_fpr_to_vreg）的重定向。

3. **TLS 访问方式**：ART 大量使用 TLS 存储线程状态。LoongArch 使用 $tp（$r2）寄存器作为 TLS 基址，访问方式与 x86 的段寄存器方式不同，需重新实现 Thread::Current() 的获取逻辑。

4. **跳转距离限制**：LoongArch 的 b 指令跳转范围为 ±128MB，beq 等条件分支仅 ±128KB。链接器需根据分支距离选择短跳转、中跳转（jirl）或长跳转（pcaddu18i + jirl）组合。

5. **精简指令集的代码密度问题**：LoongArch 定长 4 字节指令在语义表达上弱于 x86 变长指令。Nterp 的 128 字节限制内，复杂字节码需更频繁使用 HELPER 函数扩展。

4.2 调试工具链不完善

作为新兴架构，LoongArch 的调试工具链相比成熟架构仍有差距：

- **JIT 代码符号解析困难**：simpleperf 在解析 JIT 生成代码时存在符号丢失，需采用 –fp（Frame Pointer）方式进行栈展开，并使用 –post-unwind 选项离线分析。
- **缺乏性能基线**：由于缺少成熟的 LoongArch Android 设备，难以建立可靠的跨平台性能对比基线，当前主要与同架构优化前后进行对比。
- **异常路径调试复杂**：编译代码的异步异常涉及 Bionic signal 机制与内核交互，需在 fault_handler_loongarch64.cc 中提取 32 个通用寄存器、PC 值、fault address 等信息进行分析。

五 后续研究计划

5.1 短期目标（1-2 个月）

1. **AOT 编译器完善**：完成 dex2oat 工具的 LoongArch 后端，支持应用安装时的预编译，消除冷启动时的 JIT 编译开销。
2. **JIT 编译质量优化**：改进指令选择和调度，充分利用 LoongArch 的指令级并行能力和流水线特性。
3. **更多 Intrinsic 方法支持**：扩展数学库（Math.sin、Math.cos）、字符串操作（String.indexOf）等高频方法的 Intrinsic 实现。

5.2 中期目标（3-4 个月）

1. **向量扩展优化**：利用 LoongArch 的 LSX（128 位）和 LASX（256 位）向量扩展，加速数组操作、字符串处理和 SIMD 友好的循环。
2. **GC 优化**：针对 CMC 垃圾回收器的 userfaultfd 机制，优化内存压缩阶段的性能。
3. **Profile-Guided 优化**：利用运行时收集的热点信息指导 AOT 编译决策，提升应用启动速度。

5.3 长期目标

1. **上游贡献**：整理代码补丁，向 AOSP 社区提交，争取获得官方接受。

2. **二进制翻译兼容**：探索在 ARTOL 中引入 ARM 到 LoongArch 的动态库兼容执行机制，支持包含 ARM native 库的主流 APK。

3. **技术文档与推广**：撰写完整的移植指南和性能优化手册，探索将相关经验推广到其他国产处理器架构。

六 总结

本研究围绕 LoongArch 平台的 ART 运行时（ARTOL）移植与优化开展工作，主要取得了以下进展：

1. **完成了 ART 运行时的完整移植**：包括 C++ 解释器适配（遵循 LoongArch 调用约定，实现辅助跳板函数）、Nterp 汇编解释器移植（定义专用 ABI 规范，实现约 230 条 DEX 字节码的汇编解释逻辑）、JIT 编译器后端开发（汇编器/反汇编器、链接器、指令生成器、SlowPath 机制）以及运行时支持系统（快速路径、异常处理、垃圾回收卡表维护）。

2. **完成了系统性的正确性验证**：通过 Native 端 GTest 测试（art_compiler_tests、art_runtime_tests 等全部通过）、Target 端测试（三种执行模式下 1033 个测试用例 100% 通过）、libcore 测试集（146 个测试包全部通过），以及差分测试、CFG 可视化等调试框架，确保 ARTOL 的语义正确性与稳定性。

3. **实施了多项系统优化**：

- **JNI 编译优化**：实现专用 JNI 编译器，支持 @FastNative（延迟降低 68.6%）和 @CriticalNative（延迟降低 77.7%）的定制化跳板代码生成
- **内联优化**：将 Intrinsic 方法压缩为单条机器指令，CaffeineMark 整体得分提升 35.37%，Method 测试项提升近 5 倍
- **OSR 栈上替换**：实现热点循环的运行时切换机制，循环密集型程序效率提升 23.05%
- **DEX 验证器优化**：通过边界缓存、HashMap→Vector 替换、UTF-8 快速路径等技术，验证延迟降低 45.3%，吞吐量提升 57.3%

4. **积累了宝贵的工程经验**：在弱内存模型下的原子操作处理、缓存友好的

数据结构选择、快速路径设计等方面形成了可复用的技术洞察。

后续工作将继续完善 AOT 编译支持，探索 LoongArch 的 LSX/LASX 向量扩展优化，并进行更全面的应用兼容性测试，为推动国产处理器软件生态建设贡献力量。

参考文献

[1] Shi Y, Casey K, Ertl M A, et al. Virtual machine showdown: Stack versus registers [J]. ACM Transactions on Architecture and Code Optimization, 2008, 4(4): 2:1-2:36.

[2] Paleczny M, Vick C, Click C. The Java HotSpot™ server compiler [C]//Proceedings of the 2001 Symposium on Java™Virtual Machine Research and Technology Symposium. USENIX Association, 2001.

[3] Suganuma T, Yasue T, Kawahito M, et al. A dynamic optimization framework for a Java just-in-time compiler [C]//Proceedings of the ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications. ACM, 2001: 180-195.

[4] Kotzmann T, Wimmer C, Mössenböck H, et al. Design of the Java HotSpot™ client compiler for Java 6 [J]. ACM Transactions on Architecture and Code Optimization, 2008, 5(1).

[5] Ertl M A, Gregg D. The structure and performance of efficient interpreters [J]. Journal of Instruction-Level Parallelism, 2003, 5: 1-25.

[6] Detlefs D, Flood C, Heller S, et al. Garbage-first garbage collection [C]//Proceedings of the 4th International Symposium on Memory Management. Vancouver, BC, Canada: ACM, 2004: 37-48.

[7] Jones R, Lins R. Garbage collection: Algorithms for automatic dynamic memory management [M]. New York: John Wiley & Sons, 1996.